

蓝桥杯30天算法冲刺集训



Day-17 并查集

Note

并查集在蓝桥杯国赛中也考的较少，普通并查集较为简单必须掌握。带权并查集也不一定不考，所以也需要了解。本讲义习题较少，主要是了解带权并查集的用处。

引入

在修建道路时，为了让尽可能多的点连通，需要修建连通两个点的公路，这就需要随时询问两个点是否已经连通。

若将已经连通的点看作一个集合，那么修建一条公路的意义，就是合并两个集合，所以如何快速查询两个点是否属于同一个集合，以及快速地合并两个集合，十分重要。

集合

集合与并查集的概念

集合是由一个或多个确定的元素所构成的整体。集合中的元素有如下三个特征：

1. 确定性:一个元素要么属于集合，要么不属于集合。
2. 互异性:集合中的元素互不相同。
3. 无序性:集合中的元素没有先后顺序。

并查集是一个可以维护集合的数据结构，它能高效支持集合的基本操作：

1. 合并两个集合。
2. 查询两个指定元素是否属于同一个集合。

需要注意的是，由于计算机存储结构的限制，并查集维护的集合是离散意义下的集合，而不是广义的集合。集合中的元素是有限的。

集合的存储

1. 数组存储

存储一个集合最简单的方式就是直接用数组。我们将一个集合的所有元素按某种特定顺序存储在数组里。使用数组存储集合，可以支持较丰富的集合操作，但是维护集合的时间复杂度较高，对于几乎所有操作，单次操作的时间复杂度都是和集合大小成正比的。

在C++语言中，我们可以使用STL中的vector来实现数组存储集合。

2. 链表存储

可以模仿数组存储方式，将集合中的元素存储在一个链表里。链表一大好处是可以避免元素的复制，这对于合并操作是比较有帮助的，在定位元素所在的集合的两端后，直接将两个集合的端点相接即可合并完成。

然而，使用链表维护集合的最坏情况时间复杂度仍然是与集合大小成正比的，在时间效率上还不够优秀。

在C++语言中，我们可以使用STL中的list来实现链表存储集合。如果题目对时间效率的要求较高，也可以选择自行实现链表。

值得一提的是，假设元素总数是 n ，且仅需要支持合并和查询操作，那么上述两种方式可以采用启发式合并的技术(即每次将较小集合合并入较大集合并修改较小集合所有元素的信息)做到总时间复杂度 $O(n \log_2 n)$ 。虽然单次操作的时间复杂度较高，但是可以证明总时间复杂度是可以接受的。

3. 森林存储

这就是并查集。

并查集

因为一个元素只可能属于一个集合，所以我们可以为每一个集合选取一个代表元。于是查询两个元素是否属于同一个集合实际上就是询问两个元素所在集合的代表元是否相同。这个询问的时间复杂度可以利用数组标记优化为 $O(1)$ 。

但是合并两个集合时需要改变其中一个集合中所有元素的代表元，时间复杂度仍然非常高，如何优化呢？

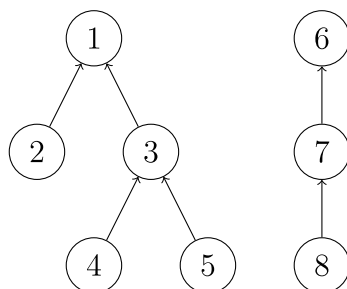
注意到，合并操作的时间复杂度远高于查询操作的复杂度，这启发我们通过一定的方式，提高查询操作的复杂度，降低合并操作的复杂度。

我们并不需要 $O(1)$ 知道每个元素所属集合的代表元，这启发我们用森林来维护代表元。用森林中的一棵树代表一个集合，树根为对应集合的代表元。

这样，对于每棵树上的元素，查询其代表元时，时间复杂度与树的高度成正比。

对两个集合进行合并操作时，只需将其中一个集合的代表元（树根）指向另一个集合（树）的代表元即可。时间复杂度也与树的高度成正比。

这就是并查集。



并查集是一种树形的数据结构，顾名思义，它用于处理一些不交集的 **合并** 及 **查询** 问题。它支持两种操作：

- 查找 (Find)：确定某个元素处于哪个子集；
- 合并 (Union)：将两个子集合并成一个集合。

定义

```
int fa[N];           // fa[i]: i 所属的集合 (i 的祖先节点)
```

初始化

```
void makeSet(int size) {  
    for (int i = 0; i < size; i++) fa[i] = i; // i就在它本身的集  
    合里  
    return;  
}
```

查找

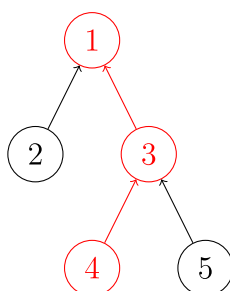
通俗地讲一个故事：几个家族进行宴会，但是家族普遍长寿，所以人数众多。由于长时间的分离以及年龄的增长，这些人逐渐忘掉了自己的亲人，只记得自己的爸爸是谁了，而最长者（称为「祖先」）的父亲已经去世，他只知道自己是谁。为了确定自己是哪个家族，他们想出了一个办法，只要问自己的爸爸是不是祖先，一层一层的向上问，直到问到祖先。如果要判断两人是否在同一家族，只要看两人的祖先是不是同一人就可以了。

在这样的思想下，并查集的查找算法诞生了。

此处给出一种 C++ 的参考实现：

```
int fa[MAXN]; // 记录某个人的爸爸是谁，特别规定，祖先的爸爸是他自己
int find(int x) {
    // 寻找 x 的祖先
    if (fa[x] == x) // 如果 x 是祖先则返回
        return x;
    else
        return find(fa[x]); // 如果不是则 x 的爸爸问 x 的爷爷
}
```

显然这样最终会返回 x 的祖先。并且上述查询操作的代码的时间复杂度取决于每个集合对应的树的高度。



合并

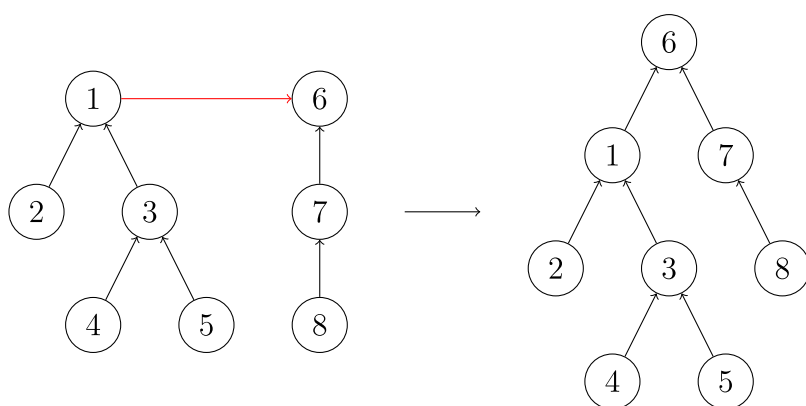
宴会上，一个家族的祖先突然对另一个家族说：我们两个家族交情这么好，不如合成一家好了。另一个家族也欣然接受了。

我们之前说过，并不在意祖先究竟是谁，所以只要其中一个祖先变成另一个祖先的儿子就可以了。

此处给出一种 C++ 的参考实现：

```
void unionSet(int x, int y) {  
    // x 与 y 所在家族合并  
    x = find(x);  
    y = find(y);  
    fa[x] = y; // 把 x 的祖先变成 y 的祖先的儿子  
}
```

可以看出，合并操作的时间复杂度取决于查找操作的时间复杂度，也就是每个集合对应的树的高度。



时间复杂度优化

不幸的是，上面代码看似优秀，但实际上，在最坏情况下时间复杂度仍然很高——因为森林中树的深度可能比较大。如果我们不停地从一个深度比较大的点向上寻找代表元，时间复杂度就令人难以接受。

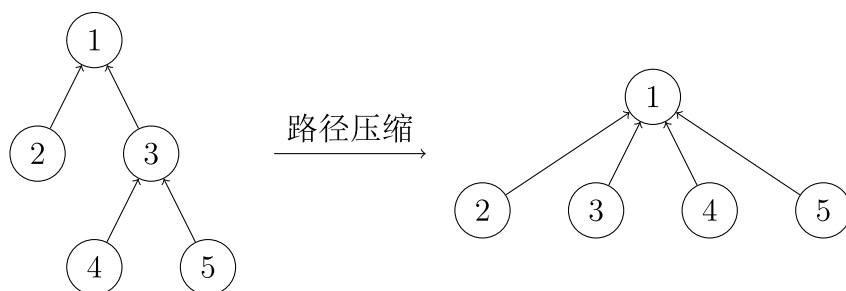
下面我们来讲讲上述操作的优化操作。

查找的优化——路径压缩

这样的确可以达成目的，但是显然效率实在太低。为什么呢？因为我们使用了太多没用的信息，我的祖先是誰与我父亲是誰没什么关系，这样一层一层找太浪费时间，不如我直接当祖先的儿子，问一次就可以出结果了。甚至祖先是誰都无所谓，只要这个人可以代表我们家族就能得到想要的效果。**把在路径上的每个节点都直接连接到根上**，这就是路径压缩。

此处给出一种 C++ 的参考实现：

```
int find(int x) {  
    if (x == fa[x]) return fa[x]; // 如果 x 是祖先(根)则返回  
    return fa[x] = find(fa[x]); // 查找 x 的祖先直到找到代表,于是  
    顺手路径压缩  
}
```



带权并查集

带权并查集的核心能力就是维护多个元素之间的连通以及偏移关系，甚至可以维护多个偏移关系。

而偏移量可以理解为当前结点到根结点的距离之和。其核心运算是向量运算。

普通的并查集仅仅记录的是集合的关系，这个关系无非是同属一个集合或者是不在一个集合。

带权并查集不仅记录集合的关系，还记录着**集合内元素的关系**或者说是集合内**元素连接线的权值**。这个权值会在查询（路径压缩）和合并时进行更新。

普通并查集本质是不带权值的图，而带权并查集则是带权的图

考虑到权值就会有以下问题：

1. 每个节点都记录的是与根节点之间的权值 $val[]$ ，那么在 Find 的路径压缩过程中，权值也应该做相应的更新，因为在路径压缩之前，每个节点都是与其父节点链接着，那个 Value 自然也是与其父节点之间的权值。
2. 在两个并查集做合并的时候，权值也要做相应的更新，因为两个并查集的根节点不同。

向量偏移法

对于集合里的任意两个元素 x, y 而言，它们之间必定存在着某种联系，因为并查集中的元素均是有联系的（这点是并查集的实质，要深刻理解），否则也不会被合并到当前集合中，那么我们就把这2个元素之间的关系量转化为一个偏移量。

路径压缩

只需要在原有代码基础上做一点修改即可。

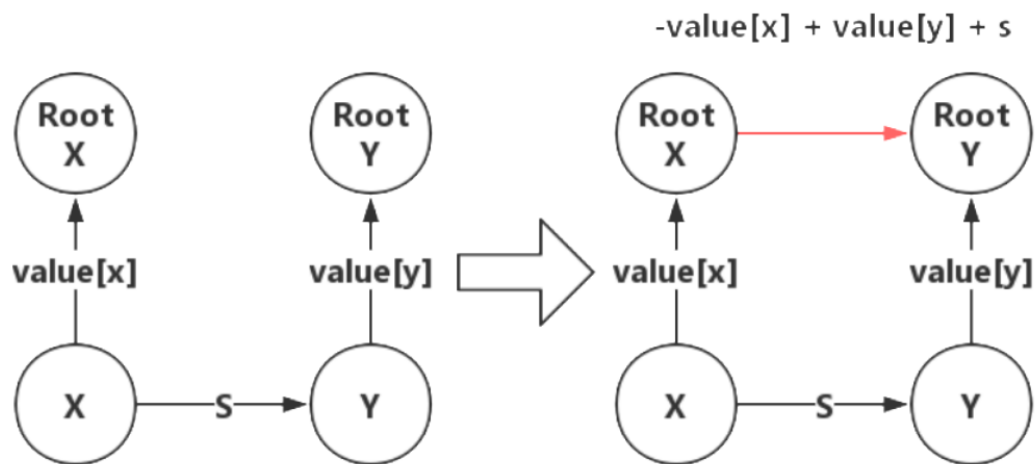
```
int fa[N];    // fa[i]: i 的父结点
int val[N];   // val[i]: 结点 i 到其所在集合的根节点的路径权值和
int find(int x) {
    if(x == fa[x]) return x;
    int k = find(fa[x]); // 记录原父节点编号
    val[x] += val[fa[x]];
    return fa[x] = k;
}
```

因为在路径压缩后父节点直接变为根节点，此时父节点的权值已经是父节点到根节点的权值了，将当前节点的权值加上原本父节点的权值，就得到当前节点到根节点的权值。

合并

已知 x, y 根节点分别为 $xRoot, yRoot$ ，如果有了 x, y 之间的关系，合并如果不考虑权值直接修改 *father* 就行了，但是现在是带权并查集，必须得求出 $xRoot$ 与 $yRoot$ 这条边的权值是多少，很显然 x 到 $yRoot$ 两条路径的权值之和应该相同，就不难得出下面代码所表达的更新式（但是需要注意并不是每个问题都是这样更新的，有时候可能会做取模之类的操作，这一点在之后的例题中可以体现）。

注意：这里的路径权值“和”在有些问题中可能会变成“乘、异或”等。



```
void merge(int x, int y, int s) {
    int xRoot = find(x);
    int yRoot = find(y);
    if (xRoot != yRoot) {
        parent[xRoot] = yRoot;
        value[xRoot] = -value[x] + value[y] + s;
    }
}
```

蓝桥杯真题

Note

因为蓝桥杯国赛中带权并查集考的较少，所以拿出省赛一道比较经典的题目进行讲解。同学们只需要理解省赛这道题目，就算入门了带权并查集了。

网络分析（蓝桥杯C/C++2020B组省赛第一场）

本题是带权并查集一道非常基础的题目，可以训练大家带权并查集的思维。

首先我们看到题目里面也明确说了要一个集合里面的才能传输数据，那么我们就把这些网线分成一个个的集合就好了。

然后就是两个集合的合并，如何进行带权并查集合并是带权并查集的一个核心的问题。

我们可以定义两个数组 $p[i]$ 代表 i 节点的父亲节点是哪个节点， $d[i]$ 代表 i 节点的权值。

在本题中我们是不是只要把路径压缩后集合唯一的一个根节点打上每次要传输信息的值就好了，后面只要查出当前集合的根节点是什么然后再把值加上去就好。

那么合并的时候是不是只需要先预处理一下把当前集合的根节点减去要合并集合的根节点的值就好了。

最后的肯定要先走一下 `find` 函数操作，也就是路径压缩与找祖先的操作，然后判断一下当前节点是不是一个区间的根节点，是的话直接输出 $d[i]$ 否则输出 $d[i]+d[\text{find}(i)]$

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

const int N = 10010;

int n, m;
int p[N], d[N];

int find(int x)
{
    if (p[x] == x || p[p[x]] == p[x]) return p[x];
    int r = find(p[x]);
    d[x] += d[p[x]];
    p[x] = r;
    return r;
}

int main()
{
    scanf("%d%d", &n, &m);
    for (int i = 1; i <= n; i++) p[i] = i;
    while (m -- )
    {
```

```

int t, a, b;
scanf("%d%d%d", &t, &a, &b);
if (t == 1)
{
    a = find(a), b = find(b);
    if (a != b)
    {
        d[a] -= d[b];
        p[a] = b;
    }
}
else
{
    a = find(a);
    d[a] += b;
}
}

for (int i = 1; i <= n; i ++ )
    if (i == find(i)) printf("%d ", d[i]);
    else printf("%d ", d[i] + d[find(i)]);
puts("");

return 0;
}

```

习题

猴子

如何建立思路，首先可以发现有一种类似桥的东西，断掉后下面所有的猴子都会掉到地上来，所以想到了并查集，一同掉下来的就是一个集合里的。

初始化 $ans[]$ 值为 ∞

然后离线做法，先加原来所有的边，然后把要断的边做个标记，然后在依此判断是否相连，得到最初的 $ans[]$ 值

然后在并查集找祖先的时候进行 $ans[]$ 值更新，即于到祖先的路径上所有点的 $ans[]$ 值最小的那个。

最后，如果 $ans[]$ 还是 ∞ ，说明他一直不会落下，就输出 -1 ，否则输出 $ans[i]$ 。

```
#include<bits/stdc++.h>
const int N = 2e5 + 5;
using namespace std;
int n, m, fa[N], son[N][3], bk[N][3], ans[N], id[2 * N], lost[2 * N];
int find(int x)
{
    int t = fa[x];
    if (fa[x] != x) fa[x] = find(fa[x]);
    ans[x] = min(ans[x], ans[t]); // 父亲掉落，儿子便掉落，用父亲更新儿子的答案
    return fa[x];
}
void Union(int x, int y, int z)
{
    int fx = find(x), fy = find(y);
    if (fx != fy)
    {
        if (fx == 1) // 保证 1 号猴子始终是父亲
            swap(fx, fy);
        fa[fx] = fy;
        ans[fx] = z;
    }
}
int main()
{
    ios::sync_with_stdio(0);
    cin >> n >> m;
    for (int i = 1; i <= n; i++)
    {
        cin >> son[i][1] >> son[i][2];
        bk[i][1] = son[i][1];
        bk[i][2] = son[i][2];
    }
}
```

```

    }
    for (int i = 1; i <= m; i++)
    {
        cin >> id[i] >> lost[i];
        son[id[i]][lost[i]] = -1;
    }
    for (int i = 1; i <= n; i++) fa[i] = i, ans[i] = 1e9;
    for (int i = 1; i <= n; i++)
    {
        if (son[i][1] != -1) Union(i, son[i][1], 1e9);
        if (son[i][2] != -1) Union(i, son[i][2], 1e9);
    }
    for (int i = m; i >= 1; i--)
    {
        int x = id[i], y = bk[x][lost[i]];
        if (y == -1) continue;
        Union(x, y, i - 1);
    }
    for (int i = 1; i <= n; i++)
    {
        find(i);
        if (ans[i] != 1e9) cout << ans[i] << '\n';
        else cout << "-1\n";
    }
}

```